

01 서비스 소개

예상 2분 · 누적 0:00 - 2:00 · 전체 원고 12분

LawPath

생활 속 법률 문제를 AI Agent가 분석하고

관련 근거를 찾아주는 서비스

발표 멘트

안녕하세요. 저희 팀은 생활 속에서 발생하는 법률 문제를, 사용자가 어려운 법률 용어를 직접 검색하지 않아도 확인할 수 있도록 돕는, LawPath라는 사이트를 개발했습니다.

지금부터 저희가 어떤 문제를 해결하려 했는지, 그리고 AI Agent와 MCP를 이용해 어떻게 구현했는지 서비스의 구조와 처리 흐름을 중심으로 설명드리겠습니다.

발표 포인트

서비스 이름과 목적을 먼저 전달한 뒤, 다음 장의 사용자 문제로 연결합니다.

02 문제 정의

예상 3분 · 누적 2:00 - 5:00

“퇴직했는데 퇴직금을 못 받았습니다.”

어떤 법을 찾아야 하지? → 어떤 판례를 봐야 하지?

→ 내 상황과 비슷한 사례는?

발표 멘트

법률 문제가 생겼을 때 가장 어려운 부분은 법률 정보가 없어서라기보다 어떤 법률 용어와 검색어로 찾아야 하는지를 모른다는 점이라고 생각했습니다.

예를 들어 사용자가 “퇴직했는데 퇴직금을 받지 못했다”고 입력했을 때, 사용자가 직접 관련 법률과 판례를 찾아야 한다면 여러 검색 과정을 거쳐야 합니다.

그래서 저희는 사용자가 자신의 상황을 자연어로 입력하면 Agent가 관련 분야를 판단하고, 법령·상담사례·판례를 찾아 연결해주는 서비스를 만들었습니다.

핵심 메시지

사용자가 법률 검색어를 몰라도 자신의 상황을 자연어로 입력할 수 있도록 합니다.

03 시스템 아키텍처

예상 3분 · 누적 5:00 - 8:00

사용자 → Streamlit Frontend

HTTP → FastAPI Backend / Agent Runtime

MCP → Legal MCP / Search Tools

PostgreSQL + pgvector → 법률 검색 결과 반환

발표 멘트

전체 구조는 크게 Frontend, Backend, Legal MCP, Database 네 영역으로 분리했습니다.

Frontend는 사용자의 질문을 받고 결과를 보여주는 역할을 하고, Backend에서는 Agent의 판단과 Tool 선택을 담당합니다.

법률 데이터 검색은 Backend가 DB를 직접 조회하는 것이 아니라 Legal MCP를 통해 수행하도록 분리했습니다.

그리고 실제 법률 데이터는 PostgreSQL과 pgvector를 기반으로 검색하도록 구성했습니다.

발표자 참고

이 경로는 법률 자료 검색 기준입니다. 회원·질의 이력 등 일반 서비스 데이터 처리와 구분해 설명합니다.

04 Agent의 처리 흐름

예상 4분 · 누적 8:00 - 12:00

질문 확인 → 필요한 정보와 검색 가능 여부 판단

정보 보완이 필요하면 → 추가 질문 후 재분석

검색 진행 → Tool 선택 → MCP 법령·판례·상담사례 검색

결과 확인 → 근거 기반 답변 및 제한사항 안내

발표 멘트

이번 과목에서 단순히 검색 기능을 만드는 것보다 중요했던 부분은 Agent가 사용자 요청을 이해하고 필요한 행동을 선택하는 과정이었습니다.

그래서 저희는 질문을 분석하고, 필요한 Tool을 선택하고, 검색 결과를 받아 사용자에게 전달하는 흐름으로 구성했습니다.

특히 필요한 정보가 부족한 경우에는 무조건 검색을 실행하는 것이 아니라 추가 정보를 요청하도록 처리했습니다.

코드 대조 메모

로컬 develop 병합본 60fb526의 공통 Runtime은 Tool 선택, 결과 형식 확인, 근거 중복 제거 및 결과 부족 상태 구분을 수행합니다. 원고 도식의 “검색 조건 수정 → 자동 재검색”과 별도 “답변 검증” 단계는 이 Runtime에서 확인되지 않아 위 흐름도에서는 제외했습니다.

발표 전 실행 서버의 Agent 구현과 일치하는지 담당자에게 확인해 주세요. 이 자료는 실행 성공이나 성능 수치를 새로 검증한 보고서는 아닙니다.

참고: README.md · backend/app/agents/runtime.py

github.com/jasnok/aio-01-p2-team2